

Lenguaje PL/SQL Oracle – Parte 3

Introducción

La sentencia **SELECT INTO** genera una tabla de resultados que consta como máximo de una fila y asigna los valores de dicha fila a las variables. Esta sentencia sólo puede incorporarse en un programa de aplicación.

La sentencia **SELECT INTO** recupera datos de una o más tablas de la base de datos y asigna los valores seleccionados a variables o colecciones. En su uso predeterminado (**SELECT ... INTO**), esta sentencia recupera una o más columnas de una sola fila.

```
SELECT EXTRACT(day FROM fecha) into dia FROM dual;
```

En esta sentencia se extraerá el día de una variable **fecha** y se colocará en la variable **dia**. Podríamos utilizar el **SELECT ... INTO** para tomar el contenido de un campo y colocarlo como contenido de una variable.

Uso de Cursores en PL/SQL para acceder a Oracle Database

Un cursor es un puntero a un área privada de SQL que almacena información sobre el procesamiento de una instrucción **SELECT** o lenguaje de manipulación de datos (DML), **INSERT**, **UPDATE**, **DELETE** o **MERGE**. Oracle Database maneja la administración de cursores de sentencias DML.

En PL/SQL no se pueden utilizar directamente sentencias **SELECT** de sintaxis básica (**SELECT <lista de campos> FROM <tabla>**). Ya que estas sentencias pueden devolver una o más filas como resultado, cosa que no se puede manejar en memoria directamente. Para esto se utilizan los cursores, para gestionar las instrucciones **SELECT**. Un cursor es un conjunto de registros devuelto por una instrucción SQL. PL/SQL ofrece varias formas de definir y manipular cursores para ejecutar declaraciones **SELECT**.

Primero, podemos distinguir dos tipos de cursores:

- **Cursores implícitos.** Este tipo de cursores se utiliza para operaciones **SELECT INTO**. Se usan cuando recibir el resultado de la consulta, un único registro. Si la consulta devolviera más de un registro y hemos utilizado un cursor implícito el bloque daría un error en tiempo de ejecución.
- **Cursores explícitos.** Son los cursores que son declarados y controlados por el programador que lo configura. Se utilizan cuando la consulta devuelve un conjunto de registros. Ocasionalmente también se utilizan en consultas que devuelven un único registro por razones de eficiencia. Son más rápidos.

Un cursor se define como cualquier otra variable de PL/SQL y debe nombrarse de acuerdo con los mismos convenios que cualquier otra variable.

Cursores Implícitos

Los cursores implícitos se utilizan para realizar consultas **SELECT** que devuelven un único registro. Deben tenerse en cuenta los siguientes puntos cuando se utilizan cursores implícitos:

- Los cursores implícitos no deben ser declarados.
- Con cada cursor implícito debe existir la palabra clave **INTO**.
- Las variables que reciben los datos devueltos por el cursor tienen que contener el mismo tipo de dato que las columnas de la tabla.
- Los cursores implícitos solo deben devolver una única fila, una y sólo una. En caso de que se devuelva más de una fila (o ninguna fila) se producirá una **excepción**. Las más comunes son:
 - **SQL%NO_DATA_FOUND:** Se produce cuando una sentencia **SELECT** intenta recuperar datos, pero ninguna fila satisface sus condiciones. Es decir, cuando “no hay datos”.

- **SQL%TOO_MANY_ROWS:** Dado que cada cursor implícito sólo es capaz de recuperar una fila, esta **excepción** detecta la existencia de más de una fila.

PL/SQL crea implícitamente un cursor para todas las sentencias SQL de manipulación de datos sobre un conjunto de filas, incluyendo aquellas sentencias que sólo devuelven una fila.

Hay diferentes conceptos relacionados con las sentencias SELECT y los cursores que debemos entender para el buen manejo de la programación:

- Cómo utilizar la declaración SELECT-INTO
- Cómo obtener un cursor explícito
- Cómo se usan los bucles FOR de cursores
- Cómo utilizar EXECUTE IMMEDIATE para consultas dinámicas
- Cómo utilizar las variables de cursor

Cada cursor tiene atributos que nos devuelve información útil sobre el estado del cursor en la ejecución de la sentencia SQL. Cuando un cursor está abierto y los datos referenciados por la consulta SELECT cambian, estos cambios no son recogidos por el cursor.

El Cursor SELECT-INTO

SELECT-INTO ofrece la forma rápida y sencilla de obtener una sola fila de una instrucción SELECT. La sintaxis de esta declaración es la siguiente:

```
SELECT lista_select INTO lista_variables FROM resto_de_consulta;
```

En donde **resto_de_consulta** contiene la lista de tablas o vistas, la cláusula WHERE y otras cláusulas de la consulta. El número y tipos de elementos en **lista_variables** deben coincidir con los de **lista_select**.

Si la declaración SELECT identifica que obtiene más de una fila, Oracle Database generará la excepción **SQL%TOO_MANY_ROWS**. Si la declaración no identifica ninguna fila para obtener, Oracle Database generará la excepción **SQL%NO_DATA_FOUND**. Veamos algunos ejemplos del uso de SELECT-INTO:

- Obtener el apellido de un empleado específico buscado por su ID (clave principal de la tabla empleados):

```
DECLARE
    l_apellido emple.apellido%TYPE;
BEGIN
    SELECT apellido INTO l_apellido FROM emple WHERE emple_id = 138;
    DBMS_OUTPUT.put_line (l_apellido);
END;
```

Si hay una fila en la tabla de empleados con ID igual a 138, este bloque mostrará el apellido de ese empleado. Si no existe tal fila, el bloque fallará con una excepción **SQL%NO_DATA_FOUND** no controlada. Suponiendo que no se define un índice único en la columna employee_id, este bloque nunca generará la excepción **SQL%TOO_MANY_ROWS**.

- Obtener una fila completa de la tabla de empleados para un ID de empleado específico:

```
DECLARE
    l_empleado empleados %ROWTYPE;
BEGIN
    SELECT * INTO l_empleado FROM empleados WHERE emple_id = 138;
    DBMS_OUTPUT.put_line (l_empleado.apellido);
END;
```

Nuevamente, si existe un empleado para esa identificación, se mostrará el apellido. En este caso, se declara un registro basado en la tabla de empleados y se buscan todas las columnas (con SELECT *) en ese registro para la fila especificada.

- Obtener columnas de diferentes tablas vinculadas:

```
DECLARE
  l_apellido empleados.apellido%TYPE;
  l_departamento_nombre departamentos.departamento_nombre%TYPE;
BEGIN
  SELECT apellido, departamento_nombre INTO l_apellido, l_departamento_nombre
    FROM empleados e, departamentos d WHERE e.departamento_id=d.id
    AND e.emple_id=138;
  DBMS_OUTPUT.put_line (l_apellido || ' en ' || l_departamento_nombre);
END;
```

En este caso, necesito más de un valor de columna, pero no todos los valores de columna son de la misma tablas, sino de ambas. Así que declaro dos variables y busco los valores de las dos columnas en esas variables.

¿Qué sucedería si la lista de variables en la cláusula INTO no coincide en tipo o cantidad con la lista SELECT de la consulta? Veremos algunos de los mensajes de error si las listas INTO y SELECT no coinciden:

ORA-00947: not enough values (no hay suficientes valores)	La lista INTO contiene menos variables que la lista SELECT.
ORA-00913: too many values (demasiados valores)	La lista INTO contiene más variables que la lista SELECT.
ORA-06502: PL/SQL: numeric or value error (error numérico o de valor)	El número de variables en las listas INTO y SELECT coincide, pero los tipos de datos no coinciden y Oracle Database no pudo convertir implícitamente de un tipo a otro.

Los Cursores Explícitos - Obtención de Cursores

Una sentencia SELECT-INTO también se conoce como una consulta implícita, porque Oracle Database implícitamente abre un cursor para la declaración SELECT, obtiene la fila y luego cierra el cursor cuando termina de hacerlo (o cuando se genera una excepción). Como alternativa, se puede declarar explícitamente un cursor y luego realizar las operaciones de apertura, recuperación y cierre usted mismo.

En PL/SQL, con los cursores, primero se los debe declarar, luego abrirlos (ejecutando la consulta SQL), iterar por los resultados fila por fila y finalmente cerrarlos para liberar recursos:

- Utilizar OPEN para abrir el cursor
- FETCH para obtener la siguiente fila.
- FOR para recorrer los resultados fila por fila
- Utilizar atributos %FOUND y %NOTFOUND para verificar si se han recuperado filas.

Declarar un Cursor

Al igual que cualquier otra variable, el cursor se declara en la sección DECLARE. Se define el nombre que tendrá el cursor y qué consulta SELECT ejecutará. No es más que una declaración. La sintaxis básica es:

```
CURSOR nombre_cursor IS instrucción_SELECT  
CURSOR nombre_cursor(param1 tipo1, ..., paramN tipoN) IS instrucción_SELECT
```

Una vez que el cursor está declarado ya podrá ser utilizado dentro del bloque de código.

Abrir un Cursor

Antes de utilizar un cursor se debe abrir. En ese momento se ejecuta la sentencia SELECT asociada y se almacena el resultado en el área de contexto (estructura interna de memoria que maneja el cursor). El resultado se guarda en el servidor en un área de memoria interna (tablas temporales) de las cuales se va retornando cada una de las filas según se va pidiendo desde el cliente. Al abrir un cursor, un puntero señalará al primer registro, a la primera fila de la consulta. La sintaxis de apertura de un cursor es:

```
OPEN nombre_cursor;  
OPEN nombre_cursor(valor1, valor2, ..., valorN);
```

Una vez que el cursor está abierto, se podrá empezar a pedir los resultados al servidor.

Recuperar cada una de las Filas de un Cursor

Una vez que el cursor está abierto en el servidor se podrá hacer la petición de recuperación de fila. En cada recuperación sólo se accederá a una **única** fila. La sintaxis de recuperación de fila de un cursor es:

```
FETCH nombre_cursor INTO variables;
```

Podremos recuperar filas mientras la consulta SELECT tenga filas pendientes de recuperar. Para saber cuándo no hay más filas podemos consultar los siguientes atributos de un cursor:

- **%NOTFOUND** devuelve TRUE cuando la última sentencia SELECT no recuperó ninguna fila, o cuando INSERT, DELETE o UPDATE no afectan a ninguna fila
- **%FOUND** devuelve TRUE cuando la última sentencia SELECT devuelve alguna fila, o cuando INSERT, DELETE o UPDATE afectan a alguna fila
- **%ROWCOUNT** devuelve el número de filas afectadas por INSERT, DELETE o UPDATE o las filas devueltas por una sentencia SELECT
- **%ISOPEN** siempre devuelve FALSE, porque Oracle cierra automáticamente el cursor implícito cuando termina la ejecución de la sentencia SELECT

Al recuperar un registro, la información recuperada se guarda en una o varias variables. Si sólo se hace referencia a una variable, ésta se puede declarar con **%ROWTYPE**. Si se utiliza una lista de variables, cada variable debe coincidir en tipo y orden con cada una de las columnas de la sentencia SELECT.

Así lo acción más típica es recuperar filas mientras queden alguna por recuperar en el servidor. Esto lo podremos hacer a través de cualquiera de los siguientes bloques:

```
OPEN nombre_cursor;  
LOOP  
    FETCH nombre_cursor INTO variables;  
    EXIT WHEN nombre_cursor%NOTFOUND; --procesar una por una las filas hasta el fin  
END LOOP;
```



```
OPEN nombre_cursor;  
FETCH nombre_cursor INTO lista_variables;  
WHILE nombre_cursor%FOUND  
    LOOP  
        /* Procesamiento de los registros recuperados */  
        FETCH nombre_cursor INTO lista_variables;  
    END LOOP;  
CLOSE nombre_cursor;
```

```
FOR variable IN nombre_cursor LOOP  
    /* Procesamiento de los registros recuperados */  
END LOOP;
```

Cerrar un Cursor

Una vez que se han recuperado todas las filas del cursor, hay que cerrarlo para que se liberen de la memoria del servidor los objetos temporales creados. Si no cerrásemos el cursor, la tabla temporal quedaría en el servidor almacenada con el nombre dado al cursor y la siguiente vez ejecutásemos ese bloque de código, nos daría la excepción `CURSOR_ALREADY_OPEN` (cursor ya abierto) cuando intentásemos abrir el cursor. Para cerrar el cursor se utiliza la siguiente sintaxis:

```
CLOSE nombre_cursor;
```

Cuando trabajamos con cursores debemos considerar:

- Cuando un cursor está cerrado, no se puede leer.
- Cuando leemos un cursor debemos comprobar el resultado de la lectura utilizando los atributos de los cursores.
- Cuando se cierra el cursor, es ilegal tratar de usarlo.
- El nombre del cursor es un identificador, no una variable. Se utiliza para identificar la consulta, por eso no se puede utilizar en expresiones.

Supongamos que se necesita escribir un bloque que obtenga la lista de empleados en orden de salario ascendente y les otorgue una bonificación de un conjunto total de fondos llamando al procedimiento de **asignar_bono**, cuyo encabezado es:

```
PROCEDURE asignar_bono (v_emple_id IN emples.id%TYPE, bonos IN OUT INTEGER)
```

Cada vez que se llama a **asignar_bono**, el procedimiento resta la bonificación otorgada del total y devuelve ese total reducido. Cuando se agota esa cuota de bonificación, deja de buscar y confirma todos los cambios.

```
1  DECLARE  
2      l_total  INTEGER := 10000;  
3      CURSOR emple_id_cur  IS SELECT emple_id FROM empleados ORDER BY sueldo ASC;  
4      l_emple_id  emple_id_cur%ROWTYPE;  
5  BEGIN  
6      OPEN emple_id_cur ;  
7      LOOP  
8          FETCH emple_id_cur  INTO l_emple_id;  
9          EXIT WHEN emple_id_cur%NOTFOUND;  
10         asignar_bono (l_emple_id, l_total);  
11         EXIT WHEN l_total <= 0;  
12     END LOOP;  
13     CLOSE emple_id_cur;  
14 END;
```

A continuación veremos las operaciones en el bloque en los números de línea especificados:

Línea	Descripción
4	La declaración explícita del cursor. Mover la consulta de la sección ejecutable (donde debe residir SELECT-INTO) y usar la palabra clave CURSOR para declarar (dar un nombre) a esa consulta (no lleva INTO).
6	Declarar un registro basado en la fila de datos devueltos por la consulta. En este caso, sólo hay un valor de una sola columna, por lo que fácilmente podría haberse declarado l_emple_id como emple_id_cur%TYPE . Pero siempre que se use un cursor explícito, es mejor declarar un registro usando %ROWTYPE, de modo que si la lista SELECT del cursor cambia alguna vez, esa variable cambiará con ella.
8	Abre el cursor, para que las filas ahora se puedan obtener de la consulta. Nota: Este es uno paso que realiza Oracle Database con la instrucción SELECT-INTO.
9	Inicia un bucle para obtener filas.
10	Obtiene la siguiente fila para el cursor y deposita la información de esa fila en el registro especificado en la cláusula INTO. Nota: Este es un paso que realiza Oracle Database con la instrucción SELECT-INTO.
11	Si FETCH no encuentra una fila, sale del bucle.
13	Llama al procedimiento asignar_bono , que aplica la bonificación y también reduce el valor de la variable l_total en esa cantidad de bonificación.
14	Salta del bucle si se han agotado todos los fondos de bonificación.
17	Cierra el cursor. Nota: Este es un paso que realiza Oracle Database con la instrucción SELECT-INTO.

Aquí hay cuatro cosas para tener en cuenta cuando se trabaja con **cursores explícitos**:

- Si la consulta no identifica ninguna fila, Oracle Database no generará NO_DATA_FOUND. En su lugar, el atributo cursor_name%NOTFOUND devolverá TRUE.
- La consulta puede devolver más de una fila y Oracle Database no generará TOO_MANY_ROWS.
- Cuando se declara un cursor en un paquete (es decir, no dentro de un subprograma del paquete) y el cursor está abierto, permanecerá abierto hasta que lo cierre explícitamente o termine su sesión.
- Cuando el cursor se declara en una sección de declaración (y no en un paquete), Oracle Database también lo cerrará automáticamente cuando finalice el bloque en el que se declara. Sin embargo, sigue siendo una buena idea cerrar explícitamente el cursor usted mismo. Si el procedimiento que contiene el cursor se mueve a un paquete, tendrá el CLOSE necesario en su lugar.

Usando el Bucle FOR del Cursor

El bucle FOR del cursor es una extensión elegante y natural del bucle FOR numérico en PL/SQL. Con un bucle FOR numérico, el cuerpo del bucle se ejecuta una vez por cada valor entero entre los valores inicial y final, especificados en el rango. Con un bucle FOR de cursor, el cuerpo del bucle se ejecuta para cada fila devuelta por la consulta.

El siguiente bloque usa un bucle FOR de cursor para mostrar los apellidos de todos los empleados en el departamento 10:

```
BEGIN
  FOR emple_rec IN (SELECT * FROM empleados WHERE depto_id = 10) LOOP
    DBMS_OUTPUT.put_line (emple_rec.apellido);
  END LOOP;
END;
```

También se puede usar un bucle FOR de cursor con un cursor declarado explícitamente:

```
DECLARE
  CURSOR emple_en_10_cur IS SELECT * FROM empleados WHERE depto_id = 10;
BEGIN
  FOR emple_rec IN emple_en_10_cur LOOP
    DBMS_OUTPUT.put_line (emple_rec.apellido);
  END LOOP;
END;
```

Lo bueno del bucle FOR del cursor es que Oracle Database abre el cursor, declara un registro usando %ROWTYPE contra el cursor, obtiene cada fila en un registro y luego cierra el bucle cuando se han obtenido todas las filas (o el bucle termina por cualquier otra razón).

Variables de cursor

Una variable de cursor es una variable que apunta a un cursor o un conjunto de resultados. A diferencia de un cursor explícito, puede pasar una variable de cursor como argumento a un procedimiento o una función. Hay varios casos de uso excelentes para las variables de cursor, incluidos los siguientes:

- Pasar una variable de cursor de vuelta al entorno host que llamó a la unidad de programa: el conjunto de resultados se puede "consumir" para su visualización u otro procesamiento.
- Construya un conjunto de resultados dentro de una función y devuelva una variable de cursor a ese conjunto. Esto es especialmente útil cuando necesita usar PL/SQL, además de SQL, para construir el conjunto de resultados.
- Pasar una variable de cursor a una función de tabla segmentada¹: una técnica de optimización poderosa pero bastante avanzada.

Veamos la sintaxis básica para trabajar con variables de cursor e identificar situaciones en las que podría considerar usar esta función. Las variables de cursor se pueden utilizar con SQL incrustado (estático) o dinámico.

El siguiente código incluye la función **nombres_para**, que devuelve una variable de cursor que obtiene nombres de empleados o departamentos, según el argumento pasado a la función.

¹ En Oracle, una "función de tabla segmentada - particionada" (o "partitioned table function") permite dividir una tabla en segmentos para optimizar el rendimiento de consultas y operaciones de mantenimiento, especialmente en tablas grandes. Una tabla segmentada se divide en partes (segmentos o particiones) basadas en un criterio definido (rango de valores de una columna, lista de valores, etc.). Una función de tabla es una función PL/SQL que devuelve una tabla como resultado. Una función de tabla segmentada combina ambos conceptos, devolviendo una tabla que es una vista de una o más particiones de una tabla segmentada.

Beneficios: **Mejor rendimiento de consultas:** Las consultas pueden enfocarse en particiones relevantes, reduciendo el tiempo de procesamiento. **Mantenimiento más eficiente:** Se pueden realizar operaciones como eliminación o actualización de particiones sin afectar a toda la tabla. **Escalabilidad:** Permite manejar grandes volúmenes de datos de manera más efectiva.

```
1 CREATE OR REPLACE FUNCTION nombres_para (  
2     nombre_tipo IN VARCHAR2)  
3     RETURN SYS_REFCURSOR  
4 IS  
5     l_retorno SYS_REFCURSOR;  
6 BEGIN  
7     CASE nombre_tipo  
8         WHEN 'EMP'  
9         THEN  
10            OPEN l_retorno FOR  
11                SELECT apellido  
12                    FROM empleados  
13                        ORDER BY emple_id;  
14                    WHEN 'DEPT'  
15            THEN  
16                OPEN l_retorno FOR  
17                    SELECT depto_nombre  
18                        FROM departamentos  
19                        ORDER BY depto_id;  
20        END CASE;  
21  
22    RETURN l_retorno;  
23 END nombres_para;
```

A continuación se describen las operaciones de cada línea de código:

Línea(s)	Descripción
3	Devuelve un dato de tipo SYS_REFCURSOR .
5	Declara una variable de cursor que la función devolverá.
7	Utiliza una sentencia CASE basada en el valor de nombre_tipo para determinar qué consulta debe abrirse.
10-13	Abre una variable de cursor para una consulta de la tabla empleados.
16-19	Abre una variable de cursor para una consulta de la tabla departamentos.

En Oracle, SYS_REFCURSOR es un tipo de dato PL/SQL que representa un puntero a un conjunto de resultados (cursor), permitiendo que una variable pueda representar cualquier tipo de cursor sin necesidad de especificar el esquema del cursor. En detalle:

- SYS_REFCURSOR no está ligado a un tipo de dato específico de fila, lo que significa que puede representar cualquier resultado de una consulta SELECT.
- **Uso:** Se utiliza para declarar variables que pueden contener cursores, lo que permite pasar cursores entre procedimientos y funciones como parámetros.

Aquí hay un bloque que utiliza la función **nombres_para** para mostrar todos los nombres en la tabla de departamentos:


```
DECLARE
  l_nombres SYS_REFCURSOR;
  l_nombre VARCHAR2 (32767);
BEGIN
  l_nombres := nombres_para ('DEPT');

  LOOP
    FETCH l_nombres INTO l_nombre;

    EXIT WHEN l_nombres%NOTFOUND;
    DBMS_OUTPUT.put_line (l_nombre);
  END LOOP;

  CLOSE l_nombres;
END;
```

Como puede verse, toda la información sobre la consulta que se abre está oculta tras el encabezado de la función. Simplemente solicita obtener los nombres de una tabla dada. La función selecciona la sentencia SELECT apropiada, abre la variable de cursor correspondiente y devuelve la variable que apunta a ese conjunto de resultados.

Una vez abierta la variable de cursor y devuelta al bloque, se utiliza el mismo código con una variable de cursor que usaría con un cursor explícito, por ejemplo:

1. FETCH del cursor (variable) INTO a una o más variables .
2. Verificar el atributo %NOTFOUND de la variable de cursor para comprobar si he terminado de recuperar todas las filas.
3. Cerrar la variable de cursor al finalizar.

La sentencia OPEN-FOR es exclusiva de las variables de cursor y permite especificar en tiempo de ejecución, sin tener que cambiar a SQL dinámico, qué conjunto de datos se recuperará a través de la variable de cursor. Sin embargo, se puede usar OPEN-FOR con una instrucción SELECT dinámica. Aquí hay un ejemplo muy sencillo: CLOSE the cursor variable when done.

```
CREATE OR REPLACE FUNCTION numeros_de (consulta_in IN VARCHAR2)
  RETURN SYS_REFCURSOR
IS
  l_retorno SYS_REFCURSOR;
BEGIN

  OPEN l_retorno FOR consulta;

  RETURN l_retorno;
END numeros_de;
```

Y aquí hay un bloque, prácticamente idéntico al que llama **nombres_para**, arriba, que muestra todos los salarios de los empleados del departamento 10:

```
DECLARE
  l_salarios SYS_REFCURSOR;
  l_salario  NUMBER;
BEGIN
  l_salarios :=
    números_de (
      'select salario
        from empleados
        where depto_id = 10');

  LOOP
    FETCH l_salarios INTO l_salario;

    EXIT WHEN l_salarios%NOTFOUND;
    DBMS_OUTPUT.put_line (l_salario);
  END LOOP;

  CLOSE l_salarios;
END;
```

Ejercicios de Cursores

Crear los siguientes bloques en PL/SQL

1. Armar una función que con la fecha de nacimiento de una persona indique cuántos días faltan para su cumpleaños.
2. Crear una función que permita contar los empleados de un departamento específico al pasárselo como parámetro a la función. Crear el bloque anónimo para probarla.
3. Actualizar el precio de un determinado producto dado el código y el nuevo precio del producto.
4. Mostrar el precio de venta de un **artículo** indicado (crear dos funciones una que muestre el precio y la otra que el precio más IVA).
5. Mostrar cuántos Profesores ingresaron este año y cuántos alumnos cursan Ingeniería Industrial.
6. Devolver los nombres de los tres productos más vendidos.
7. Realizar el aumento a todos los productos de acuerdo con un porcentaje pedido.
8. Crear un procedimiento almacenado que devuelva la lista de la razón social de los clientes que no tienen facturas emitidas.
9. Crear un procedimiento almacenado para actualizar el precio de un producto que se envíe como parámetro de entrada junto con el código de este.
10. Hacer, por departamento, el reporte de los empleados que tienen un salario menor o igual a \$88.800,00.